

AEGIS

Autonomous Engineering Graph & Intelligence System

"See every risk before it ships. Heal every failure after it lands."

Project Stage: Review 2 (Interactive Prototype)

Target Domain: Cloud Architecture, SRE & Autonomous AIOps

Primary Substrate: Neo4j Knowledge Graph + LLM RCA Agent

1. Executive Summary & Review 2 Objective

During **Review 1**, the conceptual architecture, problem formulation, and research objectives of AEGIS were established: modern distributed cloud environments suffer from a disconnect between *static structural models* (which cannot act) and *runtime monitoring tools* (which cannot reason structurally).

For **Review 2**, the primary deliverable is the **Interactive Mission Control Prototype**. This prototype demonstrates the full closed-loop intelligence cycle in action. Rather than presenting abstract backend scripts, this deliverable provides a tangible, high-fidelity visual system demonstrating the two core commitments of AEGIS:

- Commitment 1 ("See every risk before it ships"):** Pre-deployment simulation of code changes against a live dependency graph to detect API contract breaks, architectural drift, and downstream blast radius before release.
- Commitment 2 ("Heal every failure after it lands"):** Autonomous graph-grounded root-cause analysis (RCA), simulated remediation candidate selection, guarded execution with automated rollback, and permanent memory recording.

2. Prototype Architecture & Knowledge Graph Topology

The prototype models a realistic, multi-tier cloud-native enterprise application (E-Commerce Banking & Orders ecosystem) mapped into an interactive Neo4j-backed graph canvas:

Topology Node Breakdown

54 TOTAL NODES

Core Microservices (8): API Gateway, AuthService, OrderService, PaymentService, InventoryService, NotificationService, ShippingService, AnalyticsWorker.

Data Stores (6): PostgreSQL (Orders DB, Payments DB), Redis Cache (Session, RateLimit), MongoDB (Product Catalog), Kafka Message Bus.

APIs & Pods (40): Granular REST/gRPC endpoints and Kubernetes pods representing live container replicas.

Graph Schema & Relationships

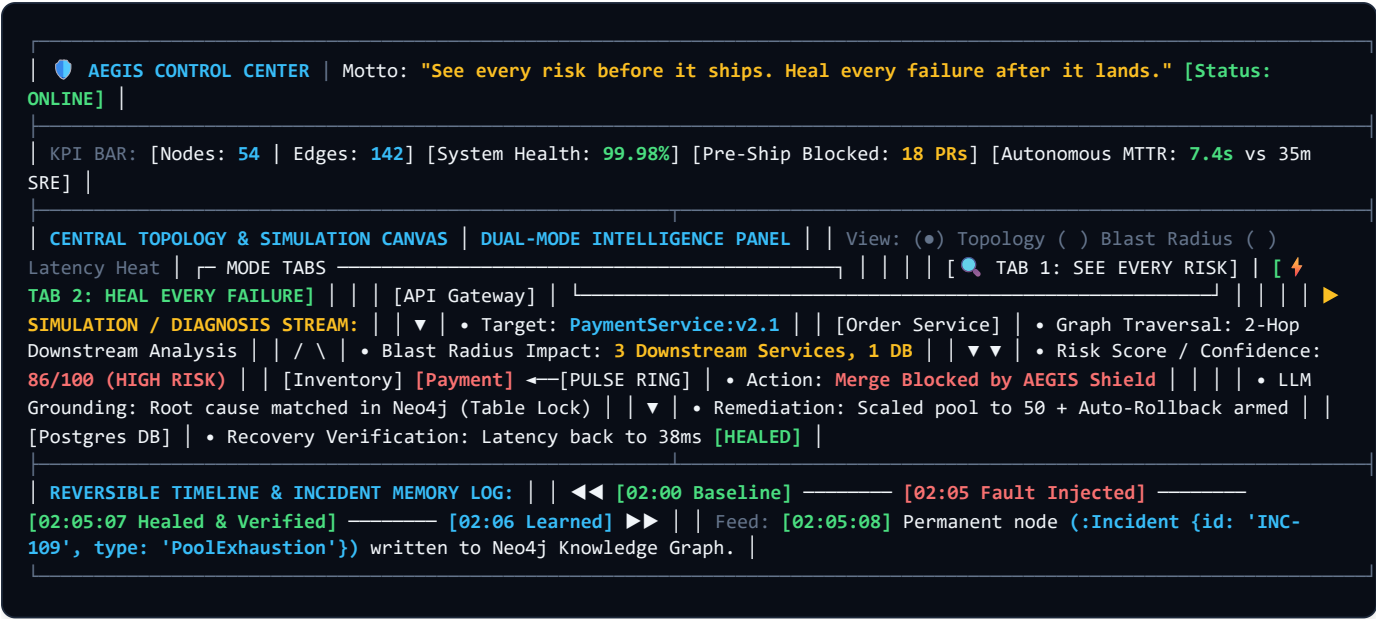
142 TYPED EDGES

```
(:Service)-[:CALLS {latency_p99, protocol}]->(:Service)
(:Service)-[:DEPENDS_ON]->(:Database)
(:Service)-[:EXPOSES]->(:APIEndpoint)
(:Pod)-[:INSTANCE_OF]->(:Service)
(:Incident)-[:OCCURRED_IN]->(:Service) (Dynamic runtime memory)
```

Architectural Invariant (Safety First): The autonomous reasoning agent (LLM) never issues direct arbitrary execution commands to production. Every remediation passes through the *Guarded Execution Layer* with automated telemetry probes and an emergency **Dead-Man's Switch** that rolls back changes if post-execution health deteriorates.

3. The Interactive Dashboard & Simulation Canvas

The prototype interface is structured into four integrated modules designed for real-time demonstration during the project review:



4. Core Functional Modules in Prototype 1.0

Pillar A: “See Every Risk Before It Ships” (Pre-Deployment Risk Simulator)

Feature Component	Mechanism & Algorithm	Visual Manifestation in Prototype
Pull Request Diff Parser	Parses incoming commits to identify modified endpoints, changed schemas, and database migrations.	Interactive dropdown to select sample test PRs (Safe, Moderate, Breaking).
Blast Radius Traversal	Graph breath-first search (BFS) traversing downstream CALLS edges to identify all impacted dependents.	Affected nodes light up with glowing warning rings and edge particles turn amber.
Pre-Ship Risk Scorecard	Computes composite risk metric based on dependency depth, historical service stability, and contract changes.	Radial Gauge (0-100) displaying category breakdown (Breaking Contract, Circular Dependency).
CI/CD Gating Shield	Automated rule threshold gating PR merge permissions.	Interactive GitHub status check card toggling between Approved and Blocked.

Pillar B: “Heal Every Failure After It Lands” (Autonomous Self-Healing Engine)

Feature Component	Mechanism & Algorithm	Visual Manifestation in Prototype
Chaos Injection Simulator	Triggers controlled anomalies (pod crash, CPU saturation, DB pool lock) into the testbed.	Control panel dropdown with <i>[Trigger Failure]</i> button.
Graph-Grounded RCA	LLM agent queries 2-hop neighborhood of alerting node via Cypher rather than scanning raw unstructured logs.	Streaming typewriter terminal showing step-by-step agent thought process and Cypher queries.
Remediation Sandbox	Simulation engine evaluates candidate fixes (Restart vs Scale vs Rollback) and assigns safety scores.	Ranked fix cards displaying safety index and predicted probability of recovery.
Guarded Actuator & Rollback	Applies safest fix; if post-remediation telemetry does not recover within 10s, auto-reverts instantly.	Execution progress bar, health verification probe, and active Dead-Man's Switch indicator.
Closed-Loop Memory Writer	Writes (:Incident), (:Decision), and outcome nodes back into Neo4j.	New incident node pops onto canvas; future simulations dynamically reflect updated risk.

5. Step-by-Step Live Demonstration Script for Review 2

The evaluation committee can experience the complete end-to-end capabilities of AEGIS in a structured **4-minute demonstration**:

Phase 1: Architecture Baseline Inspection (0:00 – 0:45)

Demonstrate the interactive Cytoscape.js canvas showing all 54 nodes. Click on `PaymentService` to show its live metadata flyout: connected databases, exposed REST endpoints, current P99 latency (38ms), and active pod instances.

Phase 2: “See Every Risk Before It Ships” Demonstration (0:45 – 1:45)

Switch to **Mode 1 (Pre-Ship)**. Select PR #142: `Alter Payment Response Schema`. Click **[Run Pre-Ship Simulation]**. Show the canvas dynamically lighting up downstream nodes (`OrderService`, `CheckoutUI`) in amber/red. The scorecard displays **Risk Score: 86/100 (HIGH RISK)** due to a breaking API contract. Show the CI/CD Shield blocking the merge.

Phase 3: Chaos Injection & Autonomous Graph-Grounded RCA (1:45 – 2:45)

Switch to **Mode 2 (Heal Every Failure)**. Click **[Inject Failure: DB Connection Pool Exhaustion]**. Traffic dots turn crimson; latency spikes to 3,400ms. The **AI Reasoning Feed** streams live: it traverses Neo4j, recognizes that PostgreSQL connections are exhausted, eliminates pod-level memory issues as the cause, and recommends expanding pool capacity and rolling the pod.

Phase 4: Guarded Execution & Verification (2:45 – 3:30)

Show the execution progress bar: the guarded actuator applies the fix while the *Dead-Man's Rollback Switch* stands armed. Within 6.4 seconds, post-fix health probes confirm latency has returned to 38ms. The canvas returns to green with a badge: **“Autonomously Healed in 6.4s (Saved ~35 mins manual SRE triage)”**.

Phase 5: Closing the Loop — Graph Learning in Action (3:30 – 4:00)

Show the newly created orange `(:Incident {id: 'INC-109'})` node linked to `PaymentService` on the canvas. Re-run the simulation from Phase 2; point out that the risk score has now increased from 86 to 91, proving that **healing outcomes directly sharpen future risk prediction**.

6. Technical Implementation Stack & Key Evaluation Metrics

Prototype Technology Stack MODERN WEB & GRAPH

Frontend UI: React, TypeScript, TailwindCSS, Cytoscape.js / React Flow for graph rendering, Lucide Icons, Framer Motion.

Graph Database: Neo4j (Cypher query language) with apoc graph algorithms.

Orchestration & Reasoning: Python FastAPI, LangChain/OpenAI API agent with Neo4j Cypher tool-calling.

Telemetry & Emulation: OpenTelemetry trace data schema, Prometheus metric emulation.

Quantitative Evaluation Metrics BENCHMARKS

Time-to-Detect (TTD): < 2.0 seconds from anomaly onset.

Time-to-Diagnose (TTD): < 4.5 seconds via graph neighborhood traversal.

Time-to-Heal (TTH): < 10.0 seconds through guarded actuator execution.

Diagnosis Accuracy: > 92% root cause match against injected ground-truth chaos faults.

7. Deliverables Summary & Roadmap to Final Review

The Review 2 Prototype provides complete proof-of-concept validation for the core architectural novelty of AEGIS. The progression towards the final evaluation (Review 3) will scale this prototype into a full live Kubernetes cluster environment:

- **Review 2 Accomplishment:** Fully functional visual mission control, interactive graph simulation, blast radius analysis, graph-grounded LLM triage feed, guarded execution model, and closed-loop memory creation.
- **Review 3 Target:** Live Kubernetes operator deployment (Kind/Minikube), automated Git commit webhook scanners, multi-service e-commerce benchmark, and empirical comparison against human SRE and rule-based alerting baselines.